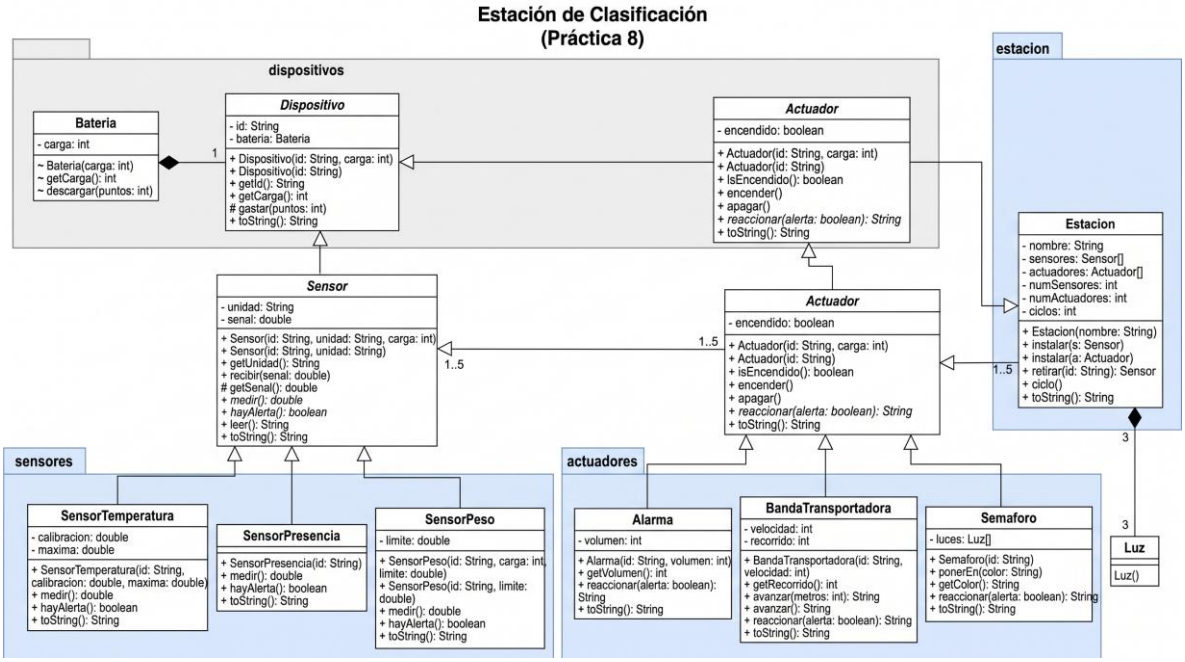
Bitácora - Práctica 8: Herencia, segunda parte

Integrantes:

- Isaac Cruz Chavez - 424020705

1. Diagrama de clases*(Inserta aquí la imagen exportada de draw.io)*



2. Salida de la consola

```
"C:\Users\Usuario\AppData\Local\Programs\Eclipse Adoptium\jdk-21.0.12.101-hotspot\bin\java.exe" "-javaagent:C:\Users\Usuario\AppData\Local\Programs\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=57967" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath "C:\Users\Usuario\IdeaProjects\Practica 8\out\production\Practica 8" estacion.Main
Instalado: Sensor S1 (100%) presencia
Instalado: Sensor S2 (80%) peso, limite 25.0
Instalado: Sensor S3 (100%) temperatura, max 45.0
Instalado: Actuador A1 (100%) banda, 2 m por ciclo
Instalado: Actuador A2 (100%) semaforo en apagado
Instalado: Actuador A3 (100%) alarma, volumen 10
== Ciclo 1 en Anden 3
S1 mide 12.0 cm
S2 mide 18.5 kg
S3 mide 40.0 grados
A1 avanza 2 m, lleva 2 m
A2 en verde
A3 en silencio
== Ciclo 2 en Anden 3
S1 mide 9.5 cm
S2 mide 32.0 kg ALERTA
S3 mide 42.0 grados
A1 se detiene
A2 en rojo
A3 suena a volumen 10
Retirado: S2
== Ciclo 3 en Anden 3
S1 mide 15.0 cm
S3 mide 46.5 grados ALERTA
A1 se detiene
A2 en rojo
A3 suena a volumen 10
Regresa: Sensor S2 (78%) peso, limite 25.0
Instalado: Sensor S2 (78%) peso, limite 25.0
A1 avanza 5 m, lleva 7 m
Anden 3: 3 sensores, 3 actuadores, 3 ciclos

Process finished with exit code 0
```

3. Respuestas teóricas

Sobrecarga contra sobrescritura

- **Sobrecarga:** Ocurre cuando una misma clase tiene dos o más métodos (o constructores) con el mismo nombre, pero que reciben diferentes parámetros. Un ejemplo en el código es la clase Estacion, que tiene el método instalar(Sensor s) y también instalar(Actuador a).
- **Sobrescritura:** Ocurre cuando una clase hija redefine el comportamiento de un método abstracto o heredado de su clase padre. Un ejemplo es el método medir() en SensorPeso, que

sobrescribe el método abstracto medir() de la clase padre Sensor.

- **Diferencia:** La sobrecarga brinda opciones de uso dentro de una misma clase variando los datos que recibe, mientras que la sobrescritura cambia la lógica interna de un método que ya venía definido por herencia.

Composición contra agregación

- La Bateria es **composición** porque el dueño (Dispositivo) hace el new directamente dentro de su propio constructor. Su existencia depende totalmente del dispositivo; si el dispositivo se destruye, la batería también muere con él.
- El sensor en la Estacion es **agregación** porque los sensores se crean desde afuera (en Main) y simplemente se instalan en la estación. Esto permite que la pieza sobreviva independiente del dueño; como se vio en el ciclo 3, la báscula (S2) pudo ser retirada para mantenimiento, la estación siguió funcionando, y la báscula conservó su estado (78% de batería) para ser reingresada después.

Modificadores Al agregar la línea bascula.gastar(5); en Main, el compilador arroja este error:java: gastar(int) has protected access in dispositivos.Dispositivo

SensorPeso sí puede usar gastar() porque este método tiene el modificador protected en la clase padre Dispositivo. El modificador protected restringe el uso del método exclusivamente a las clases dentro de su mismo paquete y a sus clases hijas. Main no cumple ninguna de estas dos condiciones (vive en el paquete estacion y no hereda de Dispositivo), por lo que el compilador bloquea el acceso por seguridad.